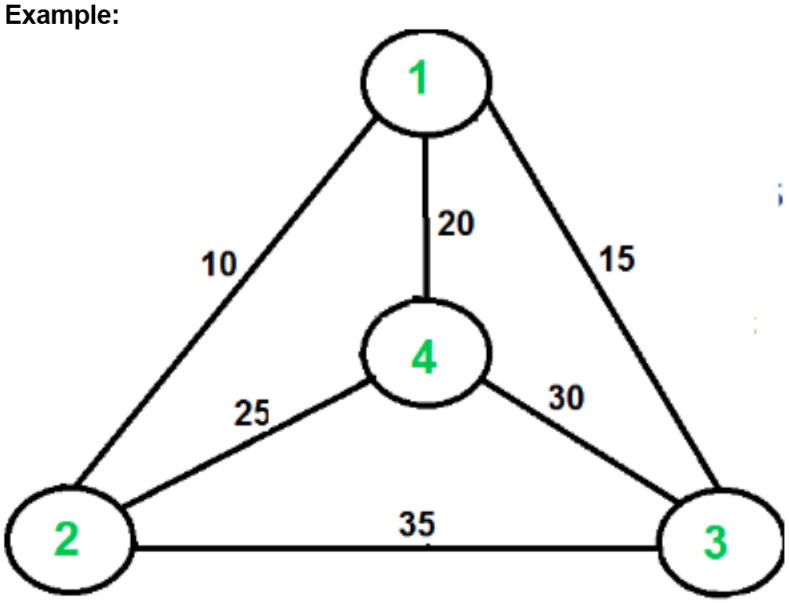
Aim:

Given a set of cities and the distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Example:



A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15, which is 80

Input Format:

- The first line contains an integer n, representing the number of cities.
- The following n lines contain the cost matrix of size $n \times n$. Each line contains n integers separated by spaces, representing the cost of traveling from one city to another. If there is no direct connection between two cities, -1 is used to represent infinity.

Output Format:

- The output is a single integer representing the optimal cost of the traveling salesman problem.

Source Code:

CTC30966.c

```
#include <stdio.h>
#include <string.h>

#define MAX_CITIES 16
#define INF 1e9

int n;
int dist[MAX_CITIES][MAX_CITIES];
int dp[1 << MAX_CITIES][MAX_CITIES];

// Bitmask DP function to find the minimum TSP cost
int tsp(int mask, int u) {
    // Base Case: If all cities have been visited, return to the start city (0)
    if (mask == (1 << n) - 1) {
        return (dist[u][0] != -1) ? dist[u][0] : INF;
```

```
    }

    // Return cached result if already calculated
    if (dp[mask][u] != -1) {
        return dp[mask][u];
    }

    int ans = INF;

    // Try visiting all other unvisited cities
    for (int v = 0; v < n; v++) {
        // Check if city v is unvisited and there is a valid path to it
        if (!(mask & (1 << v)) && dist[u][v] != -1) {
            int new_cost = dist[u][v] + tsp(mask | (1 << v), v);
            if (new_cost < ans) {
                ans = new_cost;
            }
        }
    }

    return dp[mask][u] = ans;
}

int main() {
    // Read the number of cities
    if (scanf("%d", &n) != 1) return 0;

    // Read the cost matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    // Initialize memoization table with -1
    memset(dp, -1, sizeof(dp));

    // Start at city 0, with only city 0 visited (mask = 1)
    int optimal_cost = tsp(1, 0);

    // Output the minimum cost
    printf("%d\n", optimal_cost);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
-1 2 3
4 -1 6

1 2 -2
9

Test Case - 2
User Output
4
-1 2 3 4
1 -1 1 3
4 5 -1 1
2 3 6 -1
6